

基于 AI8051U 的电子桌宠时钟系统设计

Kuri すずは

第 1 章 绪论

1.1 研究背景与意义

随着现代生活节奏加快，人们面对电脑屏幕的办公时间普遍延长，长时间的桌面工作往往容易让人感到焦虑与枯燥。针对这一问题，本设计从桌面上的传统时钟入手，发现其功能单一，仅能提供日期、时间显示及闹钟提醒，缺乏有效的人机互动。因此，本项目基于 AI8051U 设计了一款“电子桌宠时钟”，旨在通过提供互动性和情绪价值，探索传统桌面工具与趣味互动元素相结合的可行性。

1.2 作品功能简介

本作品是一个以 AI8051U 单片机为核心的桌面电子时钟系统，主要功能包括：

- 电子时钟：**支持年、月、日、时、分、秒的实时显示，可通过按键或上位机校准时间；
- 闹钟功能：**支持按键或上位机设置闹钟时间，闹钟触发后蜂鸣器响铃 30 秒，具备无应答 5 分钟后自动重响的“贪睡”功能；
- 电子桌宠：**OLED 屏幕显示像素风格的表情头像，定时自动切换表情并显示随机问候语，增强人机互动趣味性；
- 倒计时器：**可设定倒计时时间，归零时声光报警 5 秒；
- 秒表：**精确到毫秒级的计时功能，支持开始/暂停/复位；
- LED 氛围灯：**在 P0 口实现 5 种不同的 LED 氛围灯效果；
- 温度测量：**通过 NTC 热敏电阻与 ADC 采集环境温度，显示在 OLED 屏幕上；
- USB 通信：**通过 USB-CDC 虚拟串口与上位机通信，可远程设置时间和闹钟。

1.3 潜在应用领域

本作品的潜在用户主要为年轻群体，主要应用于办公及学习桌面场景。它不仅能通过精准的时间显示与多功能辅助工具（如倒计时器、秒表等）帮助用户合理规划时间，从而提高学习和工作效率，更能利用电子桌宠的趣味互动为用户带来陪伴感，在快节奏的生活与工作中起到良好的情绪调节与减压作用。

第2章 设计方案

2.1 系统构成框架图

本设计的系统构成框架图如图 1 所示。

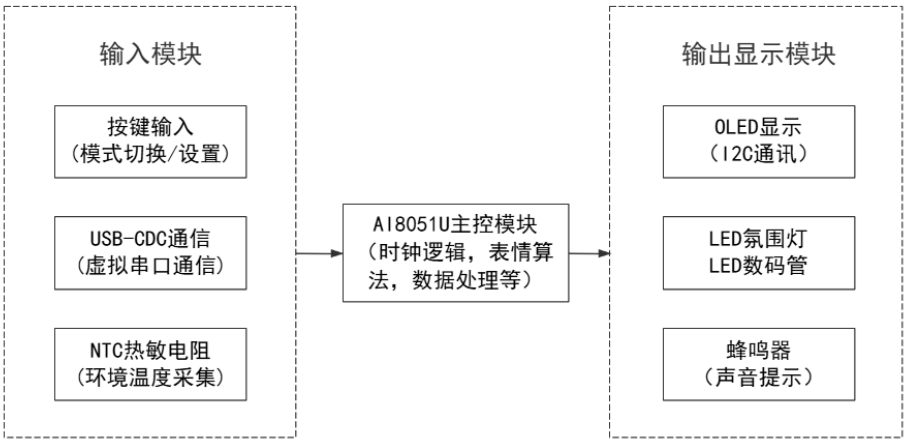


图 1 系统构成框架图

2.2 元件清单

本设计用到的电子元件如表 1 所示。

表 1 电子元件清单

序号	名称	型号与规格	数量	备注
1	单片机实验箱	AI8051U 实验箱 V1.2	1	包含按键、LED、数码管、蜂鸣器、NTC 热敏电阻等
2	OLED 显示屏	0.96 寸, I2C 接口, SSD1306,分辨率 128×64	1	——
3	跳线帽	闭口 2.54mm	2	用于连接实验箱上的功能接口

2.3 硬件设计原理

本系统的硬件设计基于 AI8051U 实验箱平台进行搭建。OLED 显示模块通过 I2C 协议进行通信，其时钟线（SCL）与数据线（SDA）分别接入单片机的 P3.2 和 P3.3 引脚；为满足系统功能需求，利用跳线帽短接 JUP2 端口以启用温度采集电路，同时短接 J1C 接口以导通蜂鸣器驱动电路。硬件系统的整体连接效果如图 2 所示。

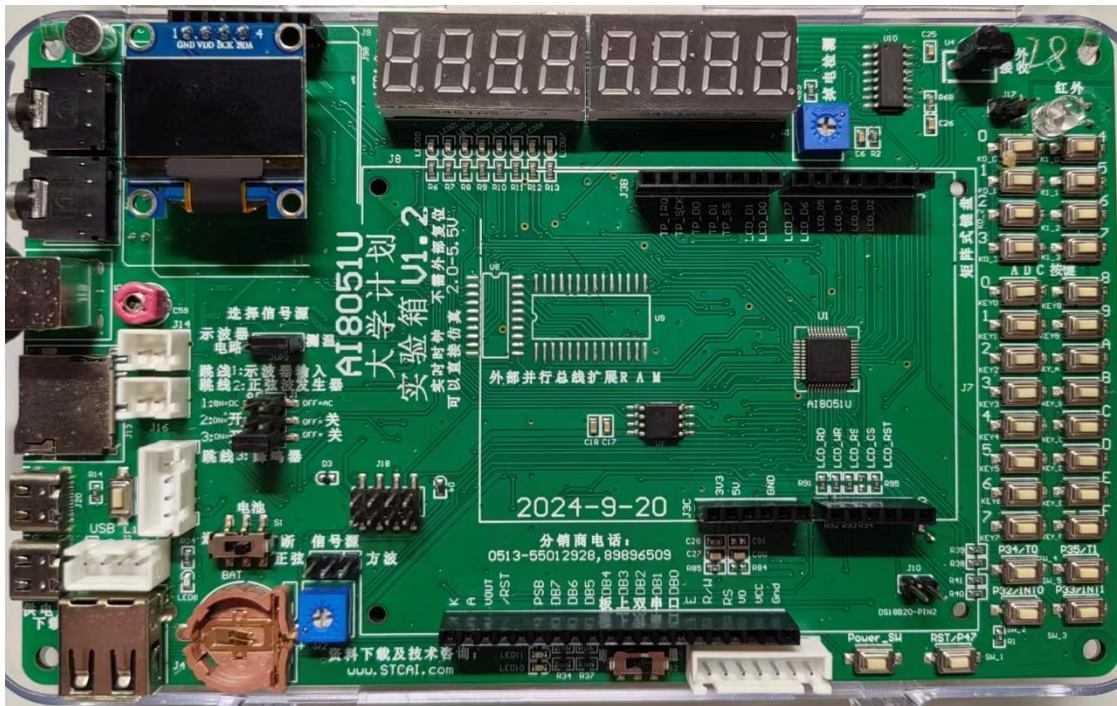
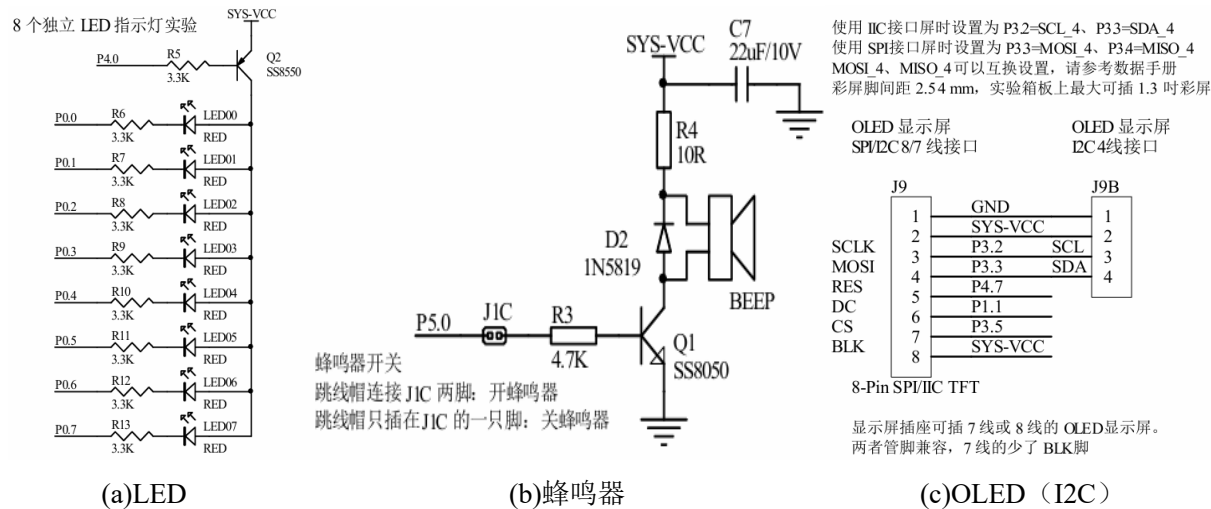
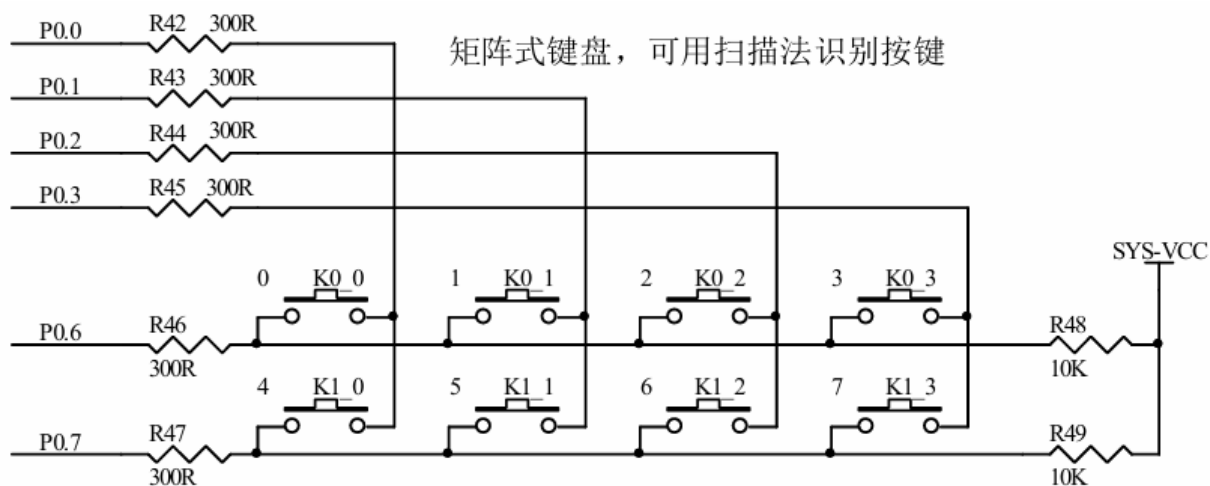


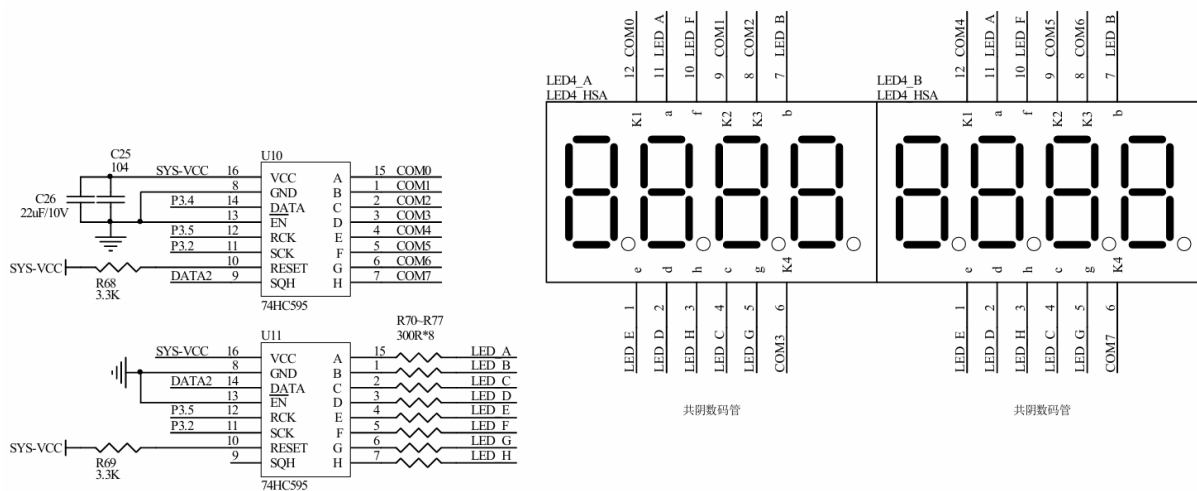
图 2 硬件总览

各功能模块的硬件电路原理图如图 3 所示，具体包括：LED 驱动电路（图 3(a))、蜂鸣器驱动电路（图 3(b))、OLED 外设接口电路（图 3(c))、矩阵式键盘电路（图 3(d)) 以及数码管及其驱动电路（图 3(e))。





(d)矩阵式键盘



(e)数码管及其驱动电路

图 3 硬件电路原理图

2.4 软件模块组成

本设计采用模块化编程思想，软件模块及其功能整理如表 2 所示。

表 2 软件模块组成

模块	文件名	功能描述
主调度模块	main.c	负责系统上电初始化，主循环任务调度（含 1ms/500ms 周期任务及空闲任务），以及 USB 通信指令的解析与处理。
RTC 时钟模块	rtc.c	维护系统实时时钟（RTC）的运行，提供时间及闹钟参数的设置、存储与读取功能。

续表

模块	文件名	功能描述
闹钟功能模块	alarm.c	负责闹钟状态的检测、响铃触发逻辑控制，以及贪睡模式的时间管理。
I2C 通信模块	i2c.c	模拟 I2C 总线时序，实现起始、停止信号及数据读写，封装 OLED 屏幕的底层复位与命令/数据接口。
OLED 显示模块	oled.c	封装 OLED 硬件初始化，实现界面绘制逻辑，包括像素图案显示及字符文字排版。
秒表功能模块	stopwatch.c	实现秒表功能的启动、暂停、复位逻辑，以及毫秒级（0.01s）计时数据的处理与显示。
LED 模块	led_effect.c	控制 P0 口 LED 的输出状态，实现 5 种氛围灯效的切换与控制。
温度采集模块	ntc.c	负责采集 NTC 热敏电阻的电压数据，并通过查表法转换为对应的温度值。
数码管显示模块	led.c	基于 74HC595 芯片驱动数码管，内置段码表，实现数字的动态扫描显示。
蜂鸣器驱动模块	beep.c	输出不同频率的方波信号，控制蜂鸣器发出不同音调的提示音。
按键模块	key.c	实现矩阵键盘的扫描检测、按键去抖动处理，以及按键事件的识别与分发。
定时器模块	timer.c	实现 Timer0/1 的初始化配置及中断服务程序（ISR），提供 1ms 系统时基与蜂鸣器驱动；同时实现倒计时器模式的计时逻辑与状态管理。

2.5 实施步骤

具体步骤及时间规划如下：

（1）方案设计与逻辑模拟阶段（5.27-6.3）

确定系统的总体设计方案，编写软件模拟 RTC 程序以验证时钟算法逻辑，并构建好时钟显示与闹钟设置的基础代码框架。

（2）底层驱动与硬件适配阶段（6.4-6.14）

基于前期的逻辑框架，将软件模拟时钟改写为硬件 RTC 驱动程序；同时编写 OLED 屏幕驱动代码，完成硬件功能测试，确保显示与计时正常。

(3) 系统集成调试阶段 (6.15-6.16)

将 RTC 模块、OLED 显示模块及测温模块集成至同一个工程中，进行多模块协同工作调试，解决硬件资源冲突与数据交互问题。

(4) 功能完善与开发阶段 (6.17-6.23)

借助 AI 辅助编程工具，快速实现按键交互逻辑、模式切换功能以及 OLED 精美 UI 界面设计，基本完成项目的全部功能开发。

(5) 总结验收阶段 (6.24-6.30)

撰写作品设计报告，对系统进行全面的功能测试与细节检查，针对发现的问题进行微调与优化。

2.6 系统流程图

项目主程序的流程图如图 4 所示。

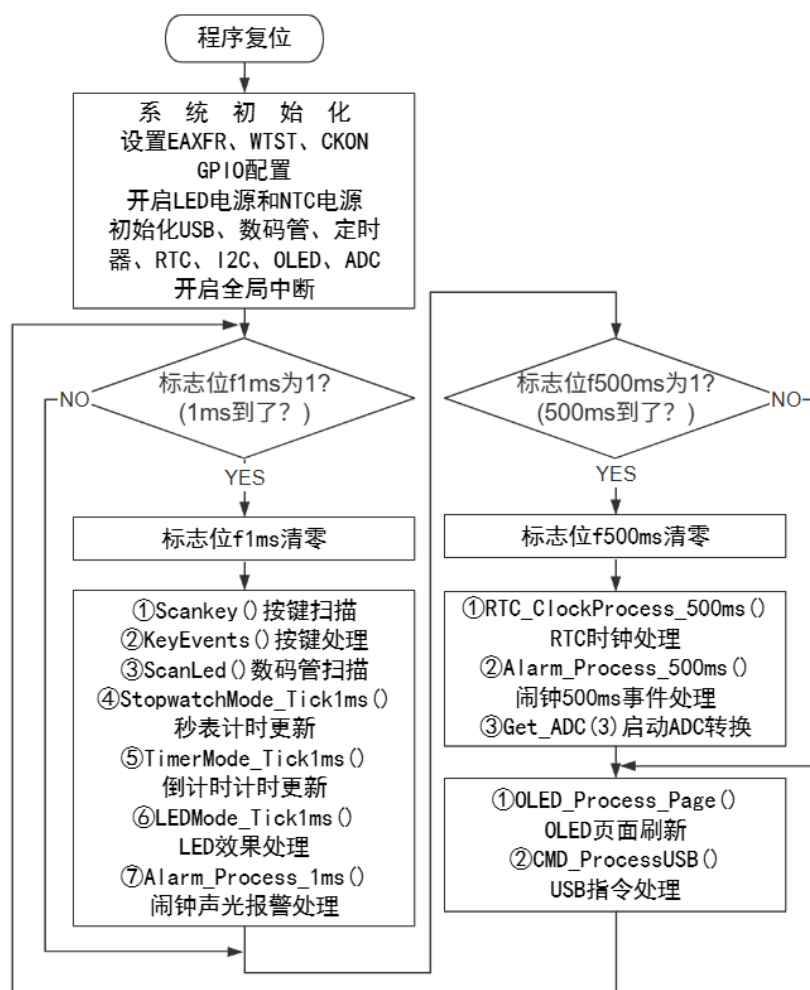


图 4 主程序流程图

2.7 各模块代码特点和创新点分析

2.7.1 OLED 主界面与桌宠显示模块 (oled.c)

本模块是作品最具特色的部分，将电子时钟与桌面宠物概念相结合，在 128×64 的 OLED 屏幕上实现了一个带有像素表情和文字对话的互动界面。

(1) 分层布局设计

屏幕采用上下分层布局，顶部为信息栏，底部为桌宠对话框：

```
//===== 顶部信息栏（页 0~页 1）=====
OLED_SetFontHeight(8);
sprintf(buf, "%04d-%02d-%02d", ...); //日期
OLED_DrawString(0, 0, buf, 0);
// ... 温度、闹钟图标、静音图标 ...

//===== 对话框（页 2~页 7）=====
OLED_DrawBox(0, 17, 127, 63, 0); //对话框边框

//===== 像素表情 =====
OLED_DrawCircle(18, 40, 11, 0); //头像圆形轮廓
OLED_DrawLine(11, 38, 16, 38, 0); //眼睛
// ... 其他面部特征 ...

//===== 对话文本 =====
OLED_SetFontHeight(16);
OLED_DrawString(40, 3, "So happy!!", 0); //问候语第一行
OLED_DrawString(40, 5, " :D ", 0); //问候语第二行
```

顶部信息栏使用 8px 小字体紧凑显示日期、温度和闹钟时间，右侧绘制 14×16 像素的闹钟和静音图标。图标根据状态不同显示不同样式：闹钟开启时显示实心铃铛，关闭时加斜杠；静音时喇叭上画叉，非静音时喇叭带有声波纹。

(2) 像素表情系统

五种表情（开心、无聊、困、爱心、酷）全部使用基本绘图函数实时绘制，不需要图片资源。以开心表情为例：

```
case 0: //开心
    OLED_DrawCircle(18, 40, 11, 0);        //圆形脸
    OLED_DrawLine(11, 38, 16, 38, 0);      //左眼（弧形上弯）
    OLED_DrawLine(20, 38, 25, 38, 0);      //右眼
    OLED_DrawPoint(13, 44, 1);             //笑容第 1 行
    OLED_DrawPoint(16, 47, 1);             //笑容底边
    OLED_DrawCircle(9, 46, 2, 0);           //左腮红
    OLED_DrawCircle(27, 46, 2, 0);          //右腮红
    break;
```

这套绘图方法完全基于 Bresenham 整数算法，避免了浮点运算，在 8 位单片机上运行效率高。

(3) 表情与问候的自动切换

表情每分钟自动切换一次，按顺序循环。每种表情配有 3 句不同的问候语，按分钟轮换显示：

```
if (bMinute != last_mood_min) {
    last_mood_min = bMinute;
    current_mood = (current_mood + 1) % 5;           //表情循环
    current_dialog = (BYTE)((bMinute + bSecond) % 3); //问候语变体
}
```

每 5 分钟完成一个表情循环，每 15 分钟完成一轮问候语循环，给人持续变化的新鲜感。

2.7.2 闹钟模块 (alarm.c 和 rtc.c 闹钟设置部分)

闹钟模块是本作品中逻辑状态最复杂的模块之一，涉及多层状态转换、多级嵌套的设置界面，以及多种停止/恢复条件。

(1) 三状态循环控制

闹钟并非简单的“到点就响”，而是在待机、响铃、贪睡三个状态之间循环，由两个标志位组合控制状态转移：

```
bit fAlarmRing = 0;           //是否正在响铃
bit fAlarmSnoozing = 0;       //是否贪睡中
BYTE bAlarmCnt = 0;           //响铃持续时间计数器（单位：500ms）
WORD wSnoozeCnt = 0;          //贪睡倒计时（600 = 5 分钟）
```

三个状态的转移逻辑在 500ms 定时中断中逐级判断：第一步检测闹钟到点信号，若闹钟已开启则进入响铃状态；第二步在响铃中累加计数值，达到 60（30 秒）后进入贪睡状态；第三步在贪睡中递减倒计时，归零后重新回到响铃状态，形成闭环。

// Alarm_Process_500ms() — 每 500ms 执行一次，处理状态转移

```
if (fAlarm) {                                     //1.闹钟到点？
    if (!fAlarmEn) { fAlarm = 0; }               //已关闭则不触发
    else if (!fAlarmRing && bMode == 0) {
        fAlarm = 0;
        fAlarmRing = 1;                          //→进入响铃
        bAlarmCnt = 0;
    }
}

if (fAlarmRing) {                                //2.正在响铃？
    if (bAlarmCnt < 60) bAlarmCnt++;              //60×500ms=30 秒
    else {
        fAlarmRing = 0;
        fAlarmSnoozing = 1;                      //→进入贪睡（5 分钟）
        wSnoozeCnt = 600;
    }
}
```

```

    }
}

if (fAlarmSnoozing) {                                //3.贪睡中?
    if (wSnoozeCnt > 0) wSnoozeCnt--;
    else {
        fAlarmSnoozing = 0;
        fAlarmRing = 1;                                //→重新响铃
        bAlarmCnt = 0;
    }
}
}

```

(2) OLED 设置界面与闪烁效果

闹钟设置通过 K1 键进入，依次经过四个步骤：ON/OFF 开关→设置小时→设置分钟→设置秒钟，最后自动保存退出。每个步骤都有专用的 OLED 画面，当前正在设置的参数位利用 fToggle（每 500ms 翻转一次）实现闪烁效果——亮半周期显示完整时间，灭半周期用空格替换当前设置位的内容，利用视觉暂留形成闪烁感：

//OLED 闹钟设置界面，当前位闪烁实现

```
blink_en = fToggle;                                // fToggle 每 500ms 翻转
```

```
if (blink_en) {
```

```
    OLED_DrawString(0, 2, "A:HH:MM:SS", 0);        // 亮半周期：正常显示
```

```
} else {
```

```
    switch (bMode) {
```

```
        case 5: OLED_DrawString(0, 2, "A:  :MM:SS", 0); break; // 灭半周期：小时隐
```

藏

```
        case 6: OLED_DrawString(0, 2, "A:HH:  :SS", 0); break; // 分钟隐藏
```

```
        case 7: OLED_DrawString(0, 2, "A:HH:MM:  ", 0); break; // 秒钟隐藏
```

```
    }
```

```
}
```

这种“隐藏式闪烁”比常见的反转色实现更简单，无需逐像素操作，只需在字符串层面替换即可，在 OLED 上效果清晰。

(3) 任意键停止与贪睡控制

闹钟响铃期间，按键处理函数在最高优先级位置检测 fAlarmRing 标志。一旦检测到响铃中有按键按下，立即停止响铃并清除贪睡计时，确保用户能快速静音：

//KeyEvents() — 闹钟响铃时任意键停止

```
if (fAlarmRing && fKeyOK && bKeyCode != 0xFF) {  
    fAlarmRing = 0;  
    fAlarmSnoozing = 0;           //用户主动停止，不再进入贪睡  
    wSnoozeCnt = 0;  
    bAlarmCnt = 0;  
    BeepOff();                   //关闭蜂鸣器  
    //继续处理该按键的正常功能（不 return）  
}
```

这里需要注意的是，响铃期间的按键同时具备两个作用：一是停止闹铃，二是执行按键本身的功能（如 K0 退出当前模式、K7 切换静音等），两部分逻辑串联执行，互不干扰。

响铃驱动方面，蜂鸣器以 La 音发声，同时 P0 口每 500ms 交替输出 0x00（全亮）和 0xFF（全灭），形成声光同步的提醒效果。

2.7.3 倒计时器模块 (timer.c)

倒计时器的设计围绕三个实际问题展开：如何让计时在后台持续运行而不中断、如何实现精确到毫秒的声光报警、如何防止运行中误触按键。

(1) 后台运行与模式隔离

倒计时器退出到主界面后仍然在后台继续走表。这一功能通过在 TimerMode_Tick1ms 函数中将计时代码放在模式守卫之前实现：

```
void TimerMode_Tick1ms(void)  
{  
    //===== 后台倒计时（先于模式守卫，始终运行）=====  
    if (b_tm_running) {  
        w_tm_tick++;  
    }
```

```

        if (w_tm_tick >= 1000) {           //累加 1000 次 = 1 秒
            w_tm_tick = 0;
            if (b_tm_sec > 0) b_tm_sec--;   //秒数减 1
            else if (b_tm_min > 0) {       //秒=0 时向分钟借位
                b_tm_min--;
                b_tm_sec = 59;
            }
            else {                         //分秒均为 0 → 结束
                b_tm_running = 0;
                b_tm_finished = 1;
                b_tm_alarm_cnt = 0;        //报警计数器归零
            }
        }
    }

    //===== 声光报警（也不受模式限制）=====
    if (b_tm_finished && b_tm_alarm_cnt < 10) {
        // ... 报警控制 ...
    }

    //===== 模式守卫：仅计时器模式才更新数码管 =====
    if (e_sys_mode != SYS_MODE_TIMER) return;
    SetLed(4, b_tm_min / 10, 0);          //显示分钟十位
    SetLed(5, b_tm_min % 10, 1);          //显示分钟个位 + 分隔点
    SetLed(6, b_tm_sec / 10, 0);          //显示秒钟十位
    SetLed(7, b_tm_sec % 10, 0);          //显示秒钟个位
}

```

函数分为三段，第一段负责计时累加（始终可用），第二段负责结束报警（始终可用），第三段负责 LED 数码管刷新（仅倒计时模式下可用）。三段分工明确，既实现了后台计时，又防止了其他模式下的显示干扰。

（2）独立计时的声光报警

倒计时归零后触发 5 秒声光报警。初版代码直接使用了系统的 500ms 翻转标志 fToggle 来控制 LED 和蜂鸣器的交替开关，但 fToggle 的相位与倒计时结束的时刻无关，导致第一段和最后一段的时长不均等。改进后的方案为报警引入了独立的毫秒累加器 w_tm_alarm_tick，从结束瞬间开始计时，确保每个半周期精确为 500ms：

```
//用 b_tm_alarm_cnt 的奇偶性控制开关状态
if (b_tm_alarm_cnt & 1) {           // 奇数→灭+静
    P0 = 0xff;
    BeepOff();
} else {                             // 偶数→亮+响
    P0 = 0x00;
    if (!b_mute) BeepOn(5);        // 静音模式下不发声
}

w_tm_alarm_tick++;
if (w_tm_alarm_tick >= 500) {       // 每半周期精确 500ms
    w_tm_alarm_tick = 0;
    b_tm_alarm_cnt++;
    if (b_tm_alarm_cnt >= 10) {     // 10 半周期 = 5 秒
        P0 = 0xff;
        BeepOff();                 // 自动停止
    }
}
```

10 个半周期依次为：亮→灭→亮→灭→亮→灭→亮→灭→亮→灭，共 5 秒。用户按 K0、K3 或 K4 键也可提前终止报警。

（3）按键防误触设计

倒计时模式下，分钟调节键（K1/K2）和秒数调节键（K5/K6）仅在空闲状态（未运行且未结束）下响应，运行中按键无效，避免计时过程中误触导致时间跳变：

```

case 1: //K1:分钟+1
    if (!b_tm_running && !b_tm_finished) { //仅空闲时可调
        if (b_tm_min < 99) b_tm_min++;
        TimerMode_Tick1ms(); //立即刷新 LED 显示
        TM_UpdateOledTime(); //立即刷新 OLED 显示
    }
    return;

```

每次按键调整后同时调用 LED 和 OLED 刷新函数，保证两种显示器的数据同步。开始/暂停键（K3）在倒计时结束后功能切换为重置，一键将时间恢复为预设值（默认为 5 分钟），无需先退出再进入。

第 3 章 调试、测试与结果

3.1 硬件调试

3.1.1 RTC 时钟模块精度测试

由于实验箱板载已焊接好的 RTC 模块，硬件调试阶段直接对其进行了长期的走时精度测试。在连续运行 4 小时的测试中，硬件 RTC 的走时误差不大于 1 秒。相比之下，此前测试的软件模拟 RTC 在同等时长下慢了约 1 分 10 秒。测试结果表明，硬件 RTC 的精度显著优于软件模拟方案，符合日常使用需求。

此外，为了实现 RTC 的断电走时功能，尝试在实验箱的电池座上安装了 CR1220 纽扣电池。尽管查阅了相关资料并尝试了多种软件编程手段，但依然无法实现“断电后无需手动重新设置日期和时间（RTC 不重复初始化）”的效果。由于难以确切排查具体是 RTC 硬件电路未能维持后台运行，亦或是软件初始化程序在上电复位时重置了时间寄存器，最终决定撤掉 CR1220 电池，放弃掉电保持功能，采用上电复位后重新校时的工作模式。

3.1.2 NTC 温度采集模块验证

NTC 温度传感器同样为实验箱板载器件。上电后，屏幕显示的温度数值位于常温范围内。通过将其显示值与我的得力（Deli）电子闹钟进行对比，两者读数相差约 2°C。考虑到缺乏标准温度计校准以及 NTC 的固有误差，认为该偏差在允许范围内。值得注意的是，为了防止示数频繁跳变影响观感，设计中将温度显示精度设定为 1°C，而非小数点后一位。为了进一步验证功能，用手指按压 NTC，屏幕读数迅速由 26°C 升至 29°C，松手后温度回落至 27°C。该动态响应测试证明测温电路工作正常，且灵敏度良好。

3.1.3 OLED 显示与引脚冲突处理

经过资料查阅与代码调试，OLED 模块已能够正常显示图像、几何图形、ASCII 字符和汉字，验证了 I2C 通信总线的可靠性。然而，在系统集成阶段发现，将 OLED 模块与数码管模块同时接入时，I2C 总线和 74HC595 移位寄存器存在引脚冲突，导致 OLED 和数码管无法正常显示。在尝试寻找实验箱上备用的 OLED 接口未果后，决定保持默认硬件连接不变，转而在软件编程层面进行优化。

但在进行软件优化后，发现修改后的驱动程序导致汉字无法正常显示。考虑到若继续深入修改底层驱动以修复汉字显示，可能会引入更多未知的 Bug，从而影响系统的整体稳定性。经过权衡，最终采取保守策略，将显示内容妥协改为全英文显示，以确保系统核心功能的可靠运行。

3.2 软件调试

3.2.1 基础时钟驱动与 RTC 迁移

在调试初期，首先完成了基于软件模拟的 RTC 程序编写。随后，为了提高计时精度，查询芯片手册后编写了基于硬件 RTC 的驱动程序，并将其移植至数码管显示模块。经测试（如 3.1.1 节所述），硬件 RTC 走时精度显著优于软件模拟方案。针对“断电后重复初始化”的问题，尝试了包括标志位判断、寻找备份寄存器及利用 EEPROM 保存 RTC 初始化标志等多种手段，但仍未能实现掉电保持功能，于是只能放弃实现“重新上电后不用手动重复设置日期和时间”的功能。

3.2.2 OLED 显示驱动调试

随后展开 OLED 模块的调试工作，通过查阅相关资料，成功实现了图像、几何图形、ASCII 字符及汉字的显示功能。调试初期遇到了屏幕整体上移一行、底部出现“雪花”噪点的问题。经过查阅资料并进行修改，成功消除了显示异常，确保了画面的稳定性。

3.2.3 系统集成与引脚冲突解决

在将各模块整合时，发现硬件 I2C 总线与 74HC595 移位寄存器（控制数码管）存在引脚冲突，导致两个显示设备无法同时工作。为此，通过查阅资料，将 I2C 通信方式改为软件模拟（GPIO 位模拟），并引入了分页处理机制（OLED_Process_Page 函数）。经调试，OLED 与数码管实现了协同显示。虽然受限于资源冲突，数码管存在每秒微弱闪烁现象且无法显示汉字，但考虑到继续修改底层代码可能引入新的不可控风险，决定保持当前状态，以优先保证系统功能的完整性。

3.2.4 功能拓展与 AI 辅助开发

在基础功能稳固后，进一步利用 AI 辅助编程工具加速开发流程，丰富了系统的应用场景。新增了 K2 倒计时器、K3 秒表计时、K6 LED 氛围灯模式及 K7 静音模式，并为闹钟功能增加了贪睡模式。同时，优化了 OLED 主界面的 UI 设计，为每种模式定制了对应的可视化界面，清晰列出各按键功能，极大地提升了人机交互的友好度和系统的可操作性。

3.3 遇到的问题及解决方案

在调试过程中遇到的主要问题整理如下：

A. 问题描述： OLED 屏幕显示内容整体向上偏移一行，且屏幕最底行出现杂乱的雪花噪点。

原因分析： 驱动程序中的寻址逻辑存在冗余，重复的定位指令导致显存指针错位，写入数据时覆盖了错误区域。

解决方案： 优化 OLED_Refresh 函数，注释掉不必要的页面及列地址定位指令，仅保留发送 1024 字节显存数据的核心操作，修复了显示错位问题。

B.问题描述：蜂鸣器发出的声音短促且断续，LED 闪烁频率异常，无法持续稳定工作。

原因分析：声光驱动逻辑放置在 500ms 的低频任务中，该任务反复调用 BeepOn()并重装定时器初值，打断了蜂鸣器方波的生成周期，导致波形破碎。

解决方案：将声光驱动逻辑移至 1ms 的主循环任务中，采用标志位进行“翻转”控制。高频刷新保证了方波周期的连续性，使发声和发光恢复正常。

C.问题描述：每次系统上电复位后，需要重新设置时间，比较麻烦。

调试尝试：尝试在 RTC 初始化处利用 if 语句判断、查找 RTC 备份寄存器、利用 EEPROM 保存初始化标志及使用 ISP 用户接口等方法。

原因分析：推测可能是实验箱电池座电路非独立给 RTC 供电，或者复位电路导致 RTC 寄存器被清零，受限于硬件资料不足，无法确切定位硬件原因。

解决方案：鉴于多种软件手段均无法实现掉电保持，且无法确认是否为 VBAT 引脚连接问题，最终决定撤销纽扣电池供电，采用手动重新设置时间的方式。

D.问题描述：整合 OLED 与数码管模块后，数码管无法点亮。改为软件 I2C 后，数码管虽能显示但出现明显闪烁。

原因分析：硬件 I2C 引脚与 74HC595 移位寄存器（控制数码管）存在硬件引脚冲突。改用软件模拟 I2C 后，I2C 通信时序干扰了数码管的动态扫描。

解决方案：关闭硬件 I2C，改用软件模拟 I2C；采用“分页碎片化刷新”策略配合“引脚抗干扰”机制，调整任务执行时序。

E.问题描述：在增加 OLED 动态画面效果后，数码管出现高频快速闪烁。

原因分析：OLED 动态效果需要频繁刷新，而引脚与数码管存在冲突。

解决方案：去除动态画面效果，将 OLED 画面恢复为静态效果。

F.问题描述：界面仿佛始终锁定在“开心”模式，无法完成正常的表情切换。仔细观察发现，在尝试切换“开心”状态以外的表情时，目标画面仅短暂闪现便自动恢复为“开心”状态。

原因分析：OLED 使用的 I2C 引脚（P32/P33）同时也启用了外部中断功能（EX0/EX1）。I2C 通信时的电平变化被误判为外部中断触发，导致程序频繁跳转，打断了状态机切换。

解决方案：禁用 EX0 和 EX1 中断，并放弃使用 P32/P33 作为交互按键。

3.4 测试结果

为了全面验证系统的功能完整性与人机交互体验，本节采用模拟用户实际操作流程的方式进行测试。

3.4.1 系统初始化与上位机通信测试

系统上电复位后的默认初始状态为：日期显示 2026-01-01，时间显示 12:00:00，闹钟设定时间为 08:00。用户可利用上位机 CDC 串口助手对上述参数进行便捷配置，具体操作界面如图 5 所示。

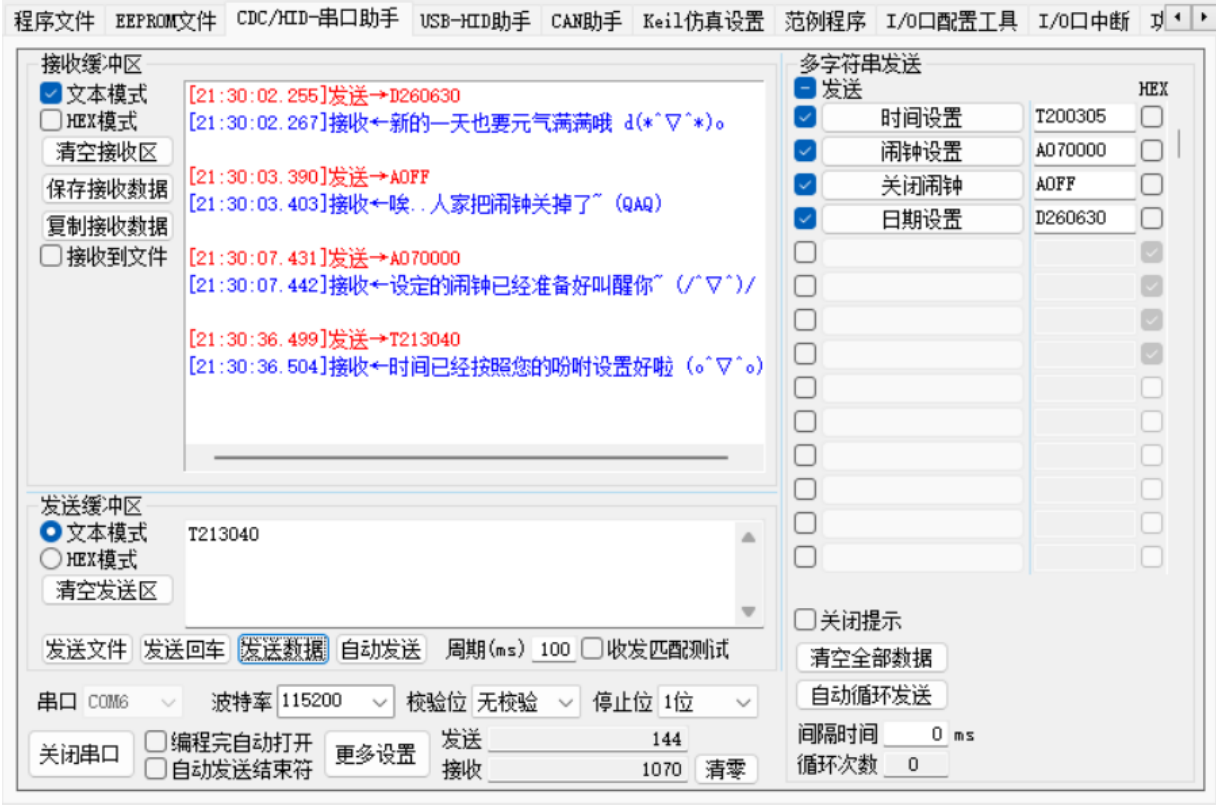


图 5 上位机 CDC 串口助手设置日期、时间和闹钟

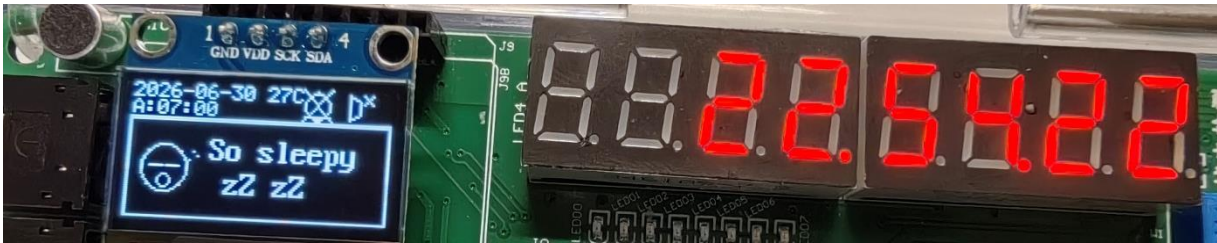
如图 5 所示，在上位机 CDC 串口助手可以快捷地设置日期、闹钟和时间。系统在接收指令后，会反馈充满互动性的中文提示语，有效丰富了人机交互的情感体验。在指令格式方面，发送“D+6 位数字”用于设置日期，发送“T+6 位数字”用于设置时间。在时间校准过程中，参照手机标准时间并合理预留指令传输的提前量，设置后的系统时间误差不超过 1 秒，具有较高的精度和操作便捷性。此外，发送“A0FF”指令可关闭闹钟功能，而发送“A+6 位数字”则可设置闹钟时间并自动开启。

3.4.2 默认显示界面

经过上述配置，产品即可正式投入使用，默认状态下的 OLED 显示与数码管显示效果如图 6 所示。



(a)示例 1



(b)示例 2

图 6 默认状态下显示效果

由图 6 可以看出，时间信息通过较大的数码管屏幕展示，方便用户频繁查看和远距离读取；而日期、闹钟时间和温度等辅助信息则集成在较小的 OLED 屏幕上，布局合理。默认状态下，OLED 屏幕上方区域展示了日期（年月日）、温度、闹钟图标及蜂鸣器图标，下方的大文本框区域则包含了随机显示的表情包和问候语。限于项目开发时间，目前实现了表情和问候语的随机显示逻辑，更复杂的显示交互逻辑及丰富效果将在后续版本中进一步优化。

3.4.3 时间与闹钟设置功能（K0 与 K1 键）

除了上位机设置，硬件按键也是系统重要的调节手段。K0 键负责进入时钟设置模式，K1 键负责进入闹钟设置模式。在没有上位机接入的场景下，用户可以通过这两个按键配合调节键完成所有时间的本地设置，保证了系统的独立可用性。

在按键交互体验上，每次按下按键都会发出按键音，且不同按键音调有所区别；长按时按键音持续发声，操作反馈直观。

1.时钟设置模式（K0 键）

连续按 K0 键，会在“小时设置”、“分钟设置”、“秒钟设置”和“正常走时”之间循环切换。处于设置状态时，对应的数字位会以 0.5 秒的频率闪烁，提示当前正在调节的内容。

2.闹钟设置模式（K1 键）

首次按下 K1 时，进入闹钟开关设置界面(如图 7 所示)。此时数码管显示“A ON”或“A OFF”，分别表示闹钟已开启或已关闭（ON/OFF 状态可通过 K4/K5 键切换）。同时 OLED 屏幕显示闹钟设置界面：顶部显示"ALARM SETTING" 标题，中间以 16px 大字体显示当前闹钟时间（格式为 A:HH:MM:SS），下方显示"ALARM ON"或 "ALARM OFF" 字样，并在右侧绘制一个小铃铛图标（闹钟开启时显示实心铃铛，关闭时不显示）。底部两行为按键提示。进入闹钟设置后，主时钟依然在后台正常走时，数码管不再显示当前时间，转而显示闹钟相关参数。



图 7 闹钟开关设置界面

再次按下 K1，进入闹钟小时设置界面（如图 8 所示）。数码管上闹钟时间的小时位以 0.5 秒为周期闪烁，提示用户当前正在调整的是小时。OLED 屏幕同步更新：中间的时间字符串中时位在闪烁半周期被空格替换，形成视觉上的闪烁效果，同时下方显示"SET HOUR"字样。用户可通过 K4/K5 键调整小时值。

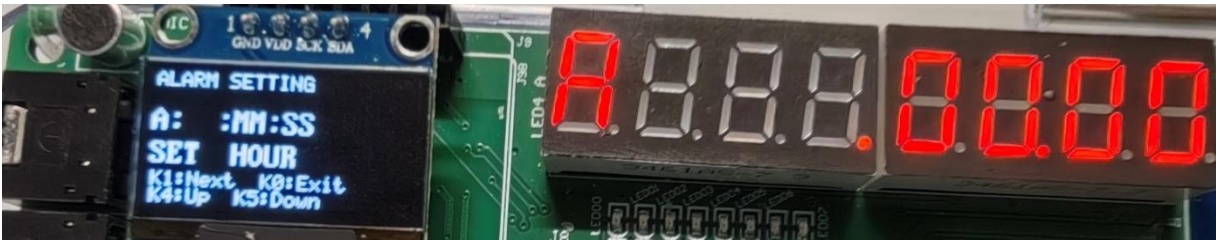


图 8 闹钟小时设置界面

继续按 K1，依次进入分钟设置和秒钟设置界面（与小时设置界面类似，故不作重复展示效果图片）。数码管和 OLED 上对应的分钟位或秒钟位同样以 0.5 秒闪烁，其余两位正常显示，下方分别提示"SET MIN"和"SET SEC"，操作方式相同。

最后一次按 K1，系统保存闹钟设置并退出到正常走时模式。此时数码管无缝切换回当前时间显示（HH:MM:SS），不会出现时间停滞或闪烁的问题，OLED 也恢复到主界面的桌宠画面。在闹钟设置的任何阶段，用户均可随时按 K0 键直接退回正常走时模式，操作灵活方便。

3.闹钟触发与贪睡机制

当系统时间与设定的闹钟时间匹配时，闹钟被触发，蜂鸣器发出 La 音提示音，同时 P0 口驱动的 LED 以 500ms 为周期交替全亮和全灭，形成声光同步的提醒效果。OLED 屏幕在闹钟响铃期间仍维持主界面显示（因定时器中断每秒检测触发，不打断当前界面操作）。数码管正常显示当前时间，不受响铃影响。

响铃持续 30 秒后，若用户未作任何操作，系统自动进入贪睡模式，蜂鸣器和 LED 停止。贪睡模式持续 5 分钟，5 分钟后闹钟自动重新响铃，再次持续 30 秒，如此循环，直到用户主动干预。

响铃期间，用户按下任意键即可立即停止闹铃，同时贪睡计时器也被清除，闹钟不会再次自动触发，避免了反复响铃的困扰。若用户希望闹钟在次日同一时间再次响起，无需重复设置，闹钟开关保持开启状态即可。闹钟的开启与关闭状态可通过 K1 键进入设置界面查看和修改，或通过 USB 指令 AOFF 远程关闭。

3.4.4 调节与操作辅助（K4 与 K5 键）

在时间设置模式（K0 进入）和闹钟设置模式（K1 进入）中，K4 和 K5 分别作为上调键（+1）和下调键（-1）使用。短按一次调整一个单位，可用于调节时钟的时、分、秒，以及闹钟的时、分、秒，也可用于切换闹钟的开启（ON）或关闭（OFF）状态。如果长按 K4 或 K5 超过 2 秒，系统会进入快速调节模式，数字以约 100ms 的间隔自动连续增减，方便大跨度调时，松开按键即停止。

3.4.5 倒计时模式（K2 键）

K2 键负责进入倒计时模式（如图 9 所示）。进入倒计时模式时，数码管左侧显示字母"C"表示倒计时模式，右侧四位显示当前的倒计时时间（格式为 MM:SS，分钟与秒之间带小数点分隔）。OLED 屏幕同步显示倒计时界面：左侧绘制闹钟图标，右侧以 16px 大字体显示倒计时时间，顶部显示"Countdown" 标题，底部两行为按键提示。进入倒计时模式后，各按键的功能自动切换为倒计时控制，其中 K2 的作用变为调整分钟数。倒计时默认时间为 5 分钟，用户可在空闲状态下通过 K1 键（分钟+1）、K2 键（分钟-1）、K5 键（秒数+1）、K6 键（秒数-1）调整倒计时时长，每次调整后数码管和 OLED 同步刷新。



图 9 倒计时模式初始界面

按下 K3 键，倒计时开始运行（如图 10 所示）。数码管上的数值逐秒递减，OLED 的倒计时时间同步更新。运行中再次按 K3 键可暂停倒计时。倒计时过程中，用户可按 K0 键退出到主界面，此时倒计时仍在后台继续运行，数码管切换回正常时间显示。再次按 K2 键进入倒计时模式时，可看到倒计时仍在继续，实现了无缝的后台计时。

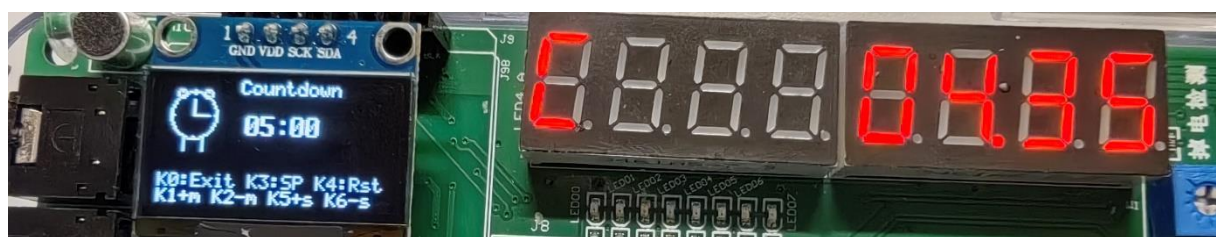


图 10 倒计时模式运行界面

倒计时归零时，触发声光报警：蜂鸣器发出 La 音，同时 P0 口驱动的 LED 全亮，两者以 500ms 为周期交替开关，持续 5 秒后自动停止。报警期间用户可按 K0、K3 或 K4 键提前终止。按 K4 键可在任意时刻将倒计时时间重置为预设值。倒计时结束后重新按 K3 键可恢复计时，按 K4 键重置到初始设定值，操作逻辑清晰连贯。

3.4.6 秒表模式（K3 键）

K3 键负责进入秒表模式（如图 11 所示）。进入秒表模式时，数码管左侧显示"St"标识秒表模式，右侧六位以 MM:SS:FF 格式显示分钟、秒钟和百分秒。OLED 屏幕显示秒表专用界面：左侧绘制一个坐在书桌前写作业的像素小人，右侧显示"Stopwatch"标题，底部为按键提示。



图 11 秒表模式初始界面

按下 K3 键，秒表开始计时（如图 12 所示），精度为百分秒（0.01 秒）。数码管上的数值持续增加，分钟最大可计至 99 分钟。再次按 K3 键可暂停计时，按 K4 键可在暂停状态下将秒表归零复位（运行中按 K4 无效，防止误触）。秒表在运行过程中，用户可按 K0 键退出到主界面，此时秒表仍在后台继续计时，数码管切换回正常时间显示。再次按 K3 键进入秒表模式时，可看到秒表仍在继续计时，与倒计时器的后台运行机制相同。



图 12 秒表模式运行界面

秒表的设计注重实用性，开始、暂停、复位三个操作分别由 K3 和 K4 两个键完成，按键复用少，操作直观。百分秒的计时精度足以满足日常计时需求，适合作为学习计时或运动计时工具。

3.4.7 LED 氛围灯模式（K6 键）

K6 键负责进入 LED 氛围灯模式（如图 13 所示）。进入 LED 氛围灯模式时，数码管仍正常显示当前时间，不受特效模式影响。OLED 屏幕显示 LED 氛围灯专用界面：顶部

显示"LED Effects" 标题，中间以 16px 大字体显示当前特效名称（如"All On"、"Blink"等），右侧显示特效编号（如"1/5"），底部为按键提示。各特效通过 P0 口控制板载 LED 实现，数码管的正常走时功能与 LED 氛围灯互不干扰，实现了显示与特效的并行工作。

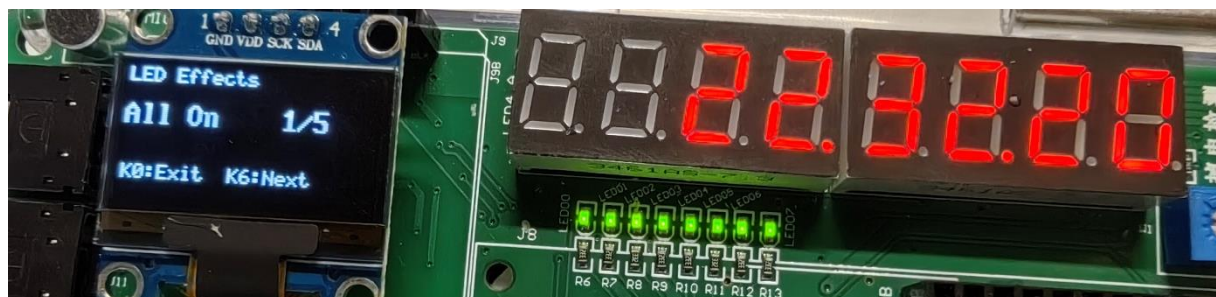


图 13 LED 氛围灯模式运行界面（全亮效果）

进入 LED 氛围灯模式后，默认从第一种效果开始。按下 K6 键，切换到下一种效果，五种效果按照"All On（全亮）→ Blink（闪烁）→ Cascade（级联灯）→ Scan（跑马灯）→ Bounce（弹球反弹）"的顺序循环切换，切换时 OLED 同步更新特效名称和编号。五种效果的具体表现为：全亮模式下所有 LED 恒定点亮；闪烁模式下 LED 以 300ms 间隔整体交替亮灭；级联模式下 LED 从右到左逐级点亮又逐级熄灭，模拟流水效果；跑马灯模式下单颗 LED 逐位循环移动；弹球模式下单颗 LED 从左到右再从右到左来回运动，模拟弹球反弹效果。在任意效果下，用户均可按 K0 键退出到主界面，退出后 P0 口恢复为默认状态，数码管保持正常走时。

3.4.8 全局静音功能（K7 键）

K7 键为全局静音开关，不属于独立的系统模式，在任何模式下均可使用。按下 K7 键，系统切换静音状态：开启静音后，按键音、闹钟响铃音、倒计时结束报警音均被静音，蜂鸣器不再发声；再次按下 K7 键，恢复所有声音输出。K7 键自身按下时不会发出按键提示音，避免切换静音时产生声音的逻辑矛盾。OLED 屏幕顶部的静音图标随状态同步更新：非静音状态显示喇叭加声波纹，静音状态显示喇叭加叉，用户可直观地了解当前是否处于静音状态。

3.5 结果误差分析与原因讨论

本系统的可能误差来源为 RTC 计时精度与 NTC 温度采集误差。

计时测试表明，在连续运行 4 小时的过程中，硬件 RTC 的累计误差控制在 1 秒以内。相比之下，此前测试的软件模拟方案在同等时长下慢了约 1 分 10 秒，硬件方案的精度提升显著。这一高精度主要得益于硬件晶振的稳定性，完全满足日常走时需求。鉴于当前短期测试误差极小，建议后续进行更长时间的连续运行测试，以进一步观察其长期漂移特性。

温度采集方面，实测显示值与得力电子闹钟读数存在约 2°C 的偏差。由于缺乏标准温度计校准，无法确定具体真实误差。产生该偏差的可能原因包括：NTC 热敏电阻的非线性特性、采样电流引起的自热效应以及基准电压的波动。此外，作为对比参照的电子闹钟本身也存在一定的测量误差。尽管存在固定偏差，但通过手指按压测试验证，传感器响应灵敏，动态变化趋势正确，整体功能符合预期。

3.6 性能指标

系统主要性能指标整理如表 3 所示。

表 3 性能指标

指标名称	参数指标
系统主频	40 MHz
计时精度	$< \pm 7 \times 10^{-5}$ （4 小时累计误差 < 1 秒）
温度测量范围	-40°C~120°C
温度测量精度	$\pm 2^{\circ}\text{C}$
OLED 界面刷新率	1Hz（1 次/秒）
数码管扫描周期	1ms/位
按键去抖时间	20 ms
蜂鸣器音调数	8 级（Do~Do 高音）
倒计时最大时长	99 分钟 59 秒
秒表最大计时	99 分 59.99 秒

第 4 章 总结

4.1 制作体会与收获

完成这个制作实属不易，从 5 月底确定了制作方案并开始逐步调试，我历经了反复的试错与攻关。其中，硬件层面的调试尤为艰难，如 I2C 总线、74HC595 移位寄存器与外部中断按键（P32/P33）之间的引脚冲突，还有 CR1220 电池无法直接帮助实现“重新上电不用重复手动设置时间”的功能等问题。排查这些问题需要消耗大量精力，也要花很多时间来思考和解决，所以有时也会感觉项目无法顺利完成，情绪时而有些低落。幸运的是，借助 AI 辅助编程工具，我的代码编写效率得到了极大提升，使得作品在功能丰富度上超出了最初的预期。尽管如此，受限于时间与技术难度，部分预想的设计细节未能最终实现，仍留有遗憾。

然而，通过此次作品的制作，我收获了很多课堂上难以获得的宝贵经验。在硬件方面，我学会了如何通过数据手册查找寄存器配置，也体会到了引脚冲突这类硬件问题往往比软件 Bug 更难定位和解决。在软件方面，我深入实践了定时器中断的配置与优先级管理、I2C 总线协议的时序模拟、矩阵键盘的扫描与去抖算法、数码管的动态扫描驱动，以及多任务状态机的设计与切换。此外，我还学会了如何利用模块化编程思想组织代码，将显示、输入、计时、通信等功能拆分为独立的模块，各自维护又协同工作。这次经历让我意识到，写出一段能运行的代码并不难，但要写出多个模块能稳定共存、互不干扰的系统，需要更全面的考量——包括中断响应时间、资源冲突、时序配合等实际工程问题。这些都是单纯看书和做题很难体会到的。

这是继实验六电子时钟系统之后，我完成的第二个综合性项目作品。本次设计的灵感来源于实验六，我希望能在保留基础功能的前提下，让电子时钟不再仅是冷冰冰的时间显示工具，而是变得更具趣味性与人文关怀，这一初衷促成了本作品的诞生。在升级过程中，我拓展了秒表、倒计时等实用工具，并创新性地引入了 LED 氛围灯及表情问候等情感交互模块。受限于开发周期，作品在功能细节与打磨度上仍有一定提升空间，但我坚信优秀作品诞生于持续的迭代，未来若有时间，我将继续对其进行深度优化。

随着课程接近尾声，回首从最初学习点亮 LED，跨越近一个学期到如今能够完成一个复杂的项目作品，这门课程让我受益匪浅，也让我对单片机开发充满了更多的热情！

4.2 改进方案

尽管本系统已实现了预设的基本功能，但在调试与使用过程中，仍发现了一些有待完善之处。针对这些不足，特制定以下改进方案：

（1）**优化 UI 界面设计**：借助 AI 绘图工具，重新设计主界面及闹钟设置、倒计时、秒表、LED 氛围灯等子模式的 OLED 页面布局，绘制更精美的 UI 图标与造型，提升视觉美感。

（2）**中文化交互提示**：探索将现有的英文问候语和提示改为中文显示，并权衡好字符长度与排版布局的关系，使交互更加符合本土化语言习惯，提升用户体验。

（3）**图标精细化重绘**：重新设计闹钟、蜂鸣器、静音等状态图标，使其线条更流畅、含义更清晰，避免因像素点过小导致的辨识困难。

（4）**增强操作指引**：优化模式界面底部的按键提示文字，使其表达更加直观易懂，帮助用户快速上手。

（5）**丰富显示逻辑**：优化表情和问候语的触发机制，使其不再局限于随机显示。改为根据时间段（整点报时）、温度（高温或低温预警）以及用户按键交互等场景动态触发不同效果，增强智能感。

（6）**升级闹铃音效**：利用现有的蜂鸣器，尝试通过编码实现旋律播放功能，将单调的“滴滴”报警声升级为活泼的音乐铃声。

（7）**完善掉电保持机制**：深入探究 RTC 硬件电路与软件初始化逻辑，解决 CR1220 电池无效的问题，真正实现上电后无需重复设置时间，提升使用的便捷性。

（8）**解决硬件冲突**：针对因引脚冲突导致的数码管闪烁问题，尝试优化软件定时器优先级或调整硬件跳线，彻底消除显示抖动。

（9）**开发闲置按键功能**：充分利用当前未使用的 ADC 按键，为其赋予新的功能（如快速切换主题、调节屏幕亮度等），丰富系统的交互维度。

（10）**构建情感系统**：设计一个“好感度/亲密度”系统，根据用户的操作习惯（如查看频率、互动次数）进行数值累加，并利用 EEPROM 进行掉电保存，高好感度可解锁隐藏表情或功能。

4.3 未来设想

在落实上述改进方案的基础上，未来还可做更加深入的拓展：

- （1）**物联网接入**：增加蓝牙或 Wi-Fi 模块（如 ESP8266），开发配套的手机 App，实现远程查看时间、无线设置闹钟、同步环境数据等功能。
- （2）**语音交互**：集成语音识别与合成模块，用户可通过语音命令控制切换模式等，单片机亦可语音播报天气或问候，实现从“触控”到“对话”的交互跨越。
- （3）**智能天气服务**：通过 Wi-Fi 连接互联网获取实时天气数据，在 OLED 上动态显示天气图标和温度范围，并结合语音功能实现天气播报。
- （4）**环境感知扩展**：外接光线传感器实现屏幕亮度自动调节；外接人体红外传感器，检测到有人靠近时自动亮屏并显示问候语，人离开后息屏省电。
- （5）**情绪自适应算法**：通过分析用户按键频率和使用模式，推测用户当前情绪状态，自动切换到对应的安慰或鼓励表情和问候语，打造有温度的电子伴侣。
- （6）**日历与日程管理**：扩展 RTC 日历功能，支持在 OLED 上查看月历，并通过 USB 或手机 App 添加日程提醒，到点时弹出文字提示。
- （7）**多用户模式**：支持多组独立的闹钟和倒计时设置，为不同家庭成员或室友设置个性化的标签与铃声，满足多人场景下的使用需求。
- （8）**多媒体功能**：外接 SD 卡模块或专用音频芯片，支持播放高品质的 MP3 格式起床铃声和背景音乐，大幅提升听觉体验。
- （9）**产品化封装**：利用 3D 打印技术制作定制的专用外壳，将电路板封装成一个结构完整、造型美观的嵌入式产品，提升作品的完成度与商业质感。
- （10）**“让桌宠动起来！”**：旨在打破屏幕的二维限制，尝试将电子桌宠从虚拟世界引向现实。通过引入舵机或步进电机构建机械躯干，赋予其头部转动、手臂摆动等拟人化动作，让它不再仅仅是一个显示图标，而是成为一个真正“活”在桌面上的立体伙伴。